CaseFiles CTF
Walkthrough

Instructor Solution Guide — All 8 Cases

Document	Walkthrough
Project	CaseFiles — Detective CTF App
Platform	Android (Kotlin + NDK)
Flag Format	CASE_CLOSED{...}
Total Cases	8
Max Score	3,300 pts

Flag Reference

#	Case Title	Technique	Points	Flag
01	The Encrypted Testimony	AES+NDK	300	`CASE_CLOSED{4e5_15_n07_s3cur3_wh3n_k3y5_4r3_3xp0s3d}`
02	The Unseen Transmission	Network Logs	300	`CASE_CLOSED{n37w0rk_sn1ff1ng_r3v34l5_s3cr375}`
03	The Unlocked Gate	Deeplink	400	`CASE_CLOSED{d33pl1nk_3xpl0173d_by_d373c71v3}`
04	The Locked Room	Frida Bypass	500	`CASE_CLOSED{fr1d4_byp4ss_m4573r_d373c71v3}`
05	The Phantom Broadcast	BroadcastReceiver	400	`CASE_CLOSED{br04dc4st_r3c31v3r_pwn3d}`
06	The Shared Secret	SharedPreferences	200	`CASE_CLOSED{sh4r3d_pr3fs_4r3_n0t_s0_pr1v4t3}`
07	The Smali Cipher	HMAC-SHA256+smali	400	`CASE_CLOSED{b4d9ad1e...306f0a}`
08	The Final Mosaic	Steganography	1000	`CASE_CLOSED{M4573R_D373C71V3_C4S3_S0LV3D_2024}`

Case 01 — The Encrypted Testimony

Technique	AES-128-CBC + NDK Binary Analysis
Points	300
Tools	jadx, Ghidra / strings, Python

Solve Steps

1. Open Case 01 in the app. The AES ciphertext is displayed on screen.
2. Extract libcasefiles.so from the APK:
```
unzip app-debug.apk -d extracted/
# .so file at: extracted/lib/arm64-v8a/libcasefiles.so
```
3. Recover the AES Key and IV from the native binary:
```
strings extracted/lib/arm64-v8a/libcasefiles.so | grep -E '.{16}'
# Key: D3t3ct1v3K3y1984
# IV:  Cr1m3Sc3n3IV1984
```

Or open in Ghidra and decompile Java_com_mystery_casefiles_crypto_NativeKeyStore_getAesKey

4. Decrypt the ciphertext using Python:
```
from Crypto.Cipher import AES
import base64
key = b'D3t3ct1v3K3y1984'
iv  = b'Cr1m3Sc3n3IV1984'
ct  = base64.b64decode('<ciphertext from app screen>')
cipher = AES.new(key, AES.MODE_CBC, iv)
plain  = cipher.decrypt(ct)
pad    = plain[-1]
print(plain[:-pad].decode())
```
5. Submit the decrypted flag.

FLAG: `CASE_CLOSED{4e5_15_n07_s3cur3_wh3n_k3y5_4r3_3xp0s3d}`

Case 02 — The Unseen Transmission

Technique	Network Interception / Logcat
Points	300
Tools	adb logcat OR Burp Suite / mitmproxy

Solve Steps

6. Open Case 02 and tap Transmit Signal.
7. Method A — Logcat (easiest):
```
adb logcat | grep BureauTransmit
# Output includes:
# I BureauTransmit: Outbound payload: Q0FTRV9DTE9TRUR7bjM3...
```
8. Method B — Burp Suite / mitmproxy:
- Configure proxy on device, install CA cert
- Capture POST to https://httpbin.org/post
- Read the 'payload' field from the JSON request body
9. Decode the Base64 payload:
```
echo 'Q0FTRV9DTE9TRUR7bjM3dzBya19zbjFmZjFuZ19yM3YzNGw1X3MzY3IzNzV9' | base64 -d
```
10. Submit the decoded flag.

FLAG: `CASE_CLOSED{n37w0rk_sn1ff1ng_r3v34l5_s3cr375}`

Case 03 — The Unlocked Gate

Technique	Insecure Deeplink Exploitation
Points	400
Tools	jadx, adb logcat, adb shell am start

Solve Steps

11. Static analysis with jadx — find SECRET_MD5 in DeeplinkChallengeActivity:
```
SECRET_MD5 = '693b3aba30b599fc88328a19fef106e0'
```
12. Find the plaintext secret via Logcat:
```
adb shell am start -n
com.mystery.casefiles/.challenges.DeeplinkChallengeActivity
adb logcat | grep BureauGate
# D BureauGate: Gate token initialised: D3t3ct1v3S3cr3t
```
13. Verify: md5('D3t3ct1v3S3cr3t') == SECRET_MD5
14. Fire the deeplink exploit:
```
adb shell am start -W \
  -a android.intent.action.VIEW \
  -d 'casefiles://open?key=D3t3ct1v3S3cr3t' \
  com.mystery.casefiles
```
15. Gate opens — flag displayed on screen. Submit it.

FLAG: `CASE_CLOSED{d33pl1nk_3xpl0173d_by_d373c71v3}`

Case 04 — The Locked Room

Technique	Frida Dynamic Instrumentation
Points	500
Tools	jadx, Frida, adb

Solve Steps

16. Static analysis (jadx) — find validatePIN() and licenseVerify() both always return false.
17. Setup Frida server on device:
```
adb push frida-server /data/local/tmp/
adb shell chmod +x /data/local/tmp/frida-server
adb shell /data/local/tmp/frida-server &
```
18. Create bypass script (bypass.js):
```
Java.perform(function() {
  var act =
Java.use('com.mystery.casefiles.challenges.FridaChallengeActivity');
  act.validatePIN.implementation  = function(p) { return true; };
  act.licenseVerify.implementation = function(l) { return true; };
});
```
19. Open Case 04 in app, then attach Frida:
```
frida -U -n com.mystery.casefiles -l bypass.js
```
20. Tap VERIFY in the app — navigates to success screen. Flag is displayed. Submit it.

FLAG: `CASE_CLOSED{fr1d4_byp4ss_m4573r_d373c71v3}`

Case 05 — The Phantom Broadcast

Technique	`Exported BroadcastReceiver Exploitation`
Points	`400`
Tools	`jadx, adb logcat, adb shell am broadcast`

Solve Steps

21. Static analysis (jadx) — open AndroidManifest.xml, find FlagReceiver exported=true.
22. Open FlagReceiver.kt — find the reversed agent code:
```
STORED_CODE_REVERSED = '7_QERF_TSOHG'
```
23. Reverse it to get the real agent code:
```
'7_QERF_TSOHG'[::-1]  =>  'GHOST_FREQ_7'
```
24. Fire the broadcast via adb:
```
adb shell am broadcast \
  -a com.mystery.casefiles.BUREAU_SIGNAL \
  -e agent_code 'GHOST_FREQ_7' \
  com.mystery.casefiles
```
25. Monitor Logcat for the flag:
```
adb logcat | grep BureauBroadcast
# I BureauBroadcast: Signal authenticated.
# I BureauBroadcast: Frequency: CASE_CLOSED{br04dc4st_r3c31v3r_pwn3d}
```
26. Submit the flag in the app.

FLAG: `CASE_CLOSED{br04dc4st_r3c31v3r_pwn3d}`

Case 06 — The Shared Secret

Technique	Insecure SharedPreferences Extraction
Points	200
Tools	adb shell run-as, Python

Solve Steps

27. Open Case 06 in the app once to seed the SharedPreferences file.
28. Extract the XML file via adb:
```
adb shell run-as com.mystery.casefiles \
  cat shared_prefs/bureau_secrets.xml
```
29. Read both values from the XML output:
```
<string name='classified_doc'>ARQBAB4WExwWBhY+Jzc...</string>
<string name='cipher_key'>BUREAU_SECRET_06</string>
```
30. Decode Base64 then XOR decrypt with the cipher_key:
```
import base64
data =
base64.b64decode('ARQBAB4WExwWBhY+JzcERHExDTUzZjkgGncgdgsxAEIdJmIaMSduJXE3YTg='
)
key  = b'BUREAU_SECRET_06'
flag = bytes([b ^ key[i % len(key)] for i, b in enumerate(data)])
print(flag.decode())
```
31. Submit the flag.

FLAG: `CASE_CLOSED{sh4r3d_pr3fs_4r3_n0t_s0_pr1v4t3}`

Case 07 — The Smali Cipher

Technique	Smali Analysis + HMAC-SHA256
Points	400
Tools	apktool, Python / CyberChef

Solve Steps

32. Decompile with apktool to get raw smali bytecode:
```
apktool d app-debug.apk -o out/
```
33. Search smali for the hardcoded salt:
```
grep -r 'D3t3ct1v3' out/smali/
# Found in SmaliCipherChallengeActivity.smali:
# const-string v0, "D3t3ct1v3S4lt9"
```
34. Read the clues displayed on screen: known message = bureau_classified, output = 64 hex chars, algorithm signs not encrypts => HMAC-SHA256.
35. Compute the flag:
```
import hmac, hashlib
salt   = b'D3t3ct1v3S4lt9'
msg    = b'bureau_classified'
result = hmac.new(salt, msg, hashlib.sha256).hexdigest()
print(f'CASE_CLOSED{{{result}}}')
```
36. Submit the flag.

FLAG:
CASE_CLOSED{b4d9ad1eb1662498756d39d784ff07d3bbe1e59545a74ccfa8a0028576306f0a}

Case 08 — The Final Mosaic

Technique	LSB Steganography
Points	1000
Tools	adb pull, Python (Pillow)

Prerequisite: Cases 01–04 must be solved first to generate the 4 evidence images.

Solve Steps

37. Pull all 4 evidence images from internal storage:
```
adb pull /data/data/com.mystery.casefiles/files/evidence/ ./evidence/
# Files: case_1.png  case_2.png  case_3.png  case_4.png
```
38. Extract LSB steganography from each image using Python:
```
from PIL import Image
import base64

def extract_lsb(path):
    img  = Image.open(path).convert('RGB')
    bits = [px[0] & 1 for px in img.getdata()]
    chars = []
    for i in range(0, len(bits)-7, 8):
        byte = int(''.join(str(b) for b in bits[i:i+8]), 2)
        if byte == 0: break
        chars.append(chr(byte))
    return base64.b64decode(''.join(chars)).decode()

parts = [extract_lsb(f'evidence/case_{i}.png') for i in range(1,5)]
print(''.join(parts))
```
39. Each image decodes to a Base64 part:

- case_1.png → Q0FTRV9DTE9TRUR7 → CASE_CLOSED{
- case_2.png → TTQ1NzNSX0QzNzND → M4573R_D373C
- case_3.png → NzFWM19DNFMzX1Mw → 71V3_C4S3_S0
- case_4.png → TFYzRF8yMDI0fQ== → LV3D_2024}

40. Concatenate all 4 decoded parts in order and submit.

FLAG: `CASE_CLOSED{M4573R_D373C71V3_C4S3_S0LV3D_2024}`

Scoring Summary

#	Case	Technique	Points	Difficulty
01	The Encrypted Testimony	AES-128-CBC + NDK	**300**	Medium
02	The Unseen Transmission	Network / Logcat	**300**	Medium
03	The Unlocked Gate	Insecure Deeplink	**400**	Medium
04	The Locked Room	Frida Bypass	**500**	Hard
05	The Phantom Broadcast	BroadcastReceiver	**400**	Medium
06	The Shared Secret	SharedPreferences	**200**	Easy
07	The Smali Cipher	HMAC-SHA256	**400**	Medium
08	The Final Mosaic	Steganography	**1000**	Hard

Total Maximum Score: 3,300 pts